



ADR-005: Outbox table with a `SKIP LOCKED` relay now; message broker later

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-005	1.0	07 October 2026	iorta TechNXT Architecture	Accepted (gaps noted below)

- **Related:** [ADR-004](#), [Solution Architecture](#), section 8

Context and problem statement

Several reactions should not run inside the caller's request:

- re-scoring after ingestion, consent change or a call outcome;
- stopping journeys and sending confirmations after a policy is issued;
- sending a renewal link the customer asked for;
- invalidating the rules cache.

These reactions must not be lost if a downstream service or replica fails. A message broker (Kafka) is not yet operated by TASCO/VETC for this workload, and adding one for the pilot means more infrastructure to run and secure.

Decision drivers

- At-least-once delivery that survives restarts.
- Run safely on N replicas without a leader.
- No new infrastructure for the pilot.
- Publishers must not change when a broker is introduced.

Considered options

1. Outbox table in Postgres (`domain_events`), relay that claims with `FOR UPDATE SKIP LOCKED` , named idempotent subscribers, dead-letter after N attempts. Forward to a broker later.
2. Kafka (or managed Pub/Sub / SNS+SQS) from day one.
3. In-process `EventEmitter` .
4. Postgres `LISTEN/NOTIFY` .

Decision outcome

Chosen option: **1**, implemented in `src/adapters/messaging/outboxEventBus.js` and `postgresStore.claimEvents` .

- `publish(type, payload, {actor, correlationId})` inserts an event `{id, type, payload, actor, correlationId, occurredAt, status: 'pending', attempts: 0}` .
- `claimEvents(limit)` runs in a short transaction: `UPDATE domain_events SET status='processing' ... WHERE id IN (SELECT id ... WHERE status='pending' OR (status='processing' AND updated_at < now() - interval '5 minutes') ORDER BY occurred_at LIMIT $1 FOR UPDATE SKIP LOCKED) RETURNING ...` . Rows abandoned in `processing` by a crashed relay are therefore reclaimed after a 5-minute lease; an index on `(status, occurred_at)` supports this (migration 002).
- `relay()` dispatches to the subscribers of that type and of `*` . It records successful handler names in `handled[]` so a retry runs only the failed ones. It stops at the first failure. With `attempts ≥ 5` the event becomes `dead_letter` , otherwise it goes back to `pending` .

- The relay is driven by three things:
 - a 1 s interval in every API replica (`src/server.js`);
 - the `relay` CronJob;
 - `events.drain()` at the end of interactive requests that need read-your-writes.

Consequences

- Good: no broker to operate. The relay scales horizontally, and events are queryable with SQL (`/api/ops/status` shows the backlog by status).
- Good: when a broker arrives, the relay becomes a forwarder (outbox → topic) and subscribers become consumers. `events.publish` call sites do not change.
- Bad / gaps (honest list):
 1. **It is not yet transactional with the state change.** `publish()` is a separate insert after the business write. A crash between the two loses the event while the state change persists. The reverse case cannot occur, because the event is written after the state. **Fix:** run use cases inside `store.transaction(tx ⇒ ...)` with a transactional event bus bound to `tx` (the port already exists).
 2. **Stuck processing rows (resolved).** The PostgreSQL adapter reclaims rows older than the 5-minute lease (see above). Keep the alert on the age of the oldest `processing` row; the in-memory store does not need a lease.
 3. **No backoff between attempts.** A failed event is retried on the next relay tick (about 1 s), so 5 attempts can be used up in seconds during a short outage. **Fix:** `next_attempt_at = now() + 2^attempts × base`.
 4. **Ordering** is by `occurred_at` per claim batch only. Handlers must not assume ordering across events. Current handlers recompute from state, so they are order-insensitive.
 5. **Growth.** `done` rows are never purged. Add `domain_events` to the retention job (for example 30 days) or use monthly partitions.
 6. Some side-effecting handlers (`send-link`, the purchase confirmation, the claim acknowledgement) create a new message id per execution, so a replay after partial failure could send twice. **Fix:** derive the message id from `eventId + channel`.

Pros and cons of the options

Kafka from day one. Durable, replayable, high-throughput. However, it is heavy to operate for pilot volumes, and it still needs an outbox to avoid dual writes.

EventEmitter. Trivial. However, events are lost on crash and it does not work across replicas.

LISTEN/NOTIFY. Low latency. However, it is not durable (notifications are lost while disconnected), so a table is still needed. It can be added later as a wake-up hint for the relay.